

Contract-Aware Quantum Opportunity Analysis: Enhancing LLMs with Program Analysis and Semantic Feedback

ANONYMOUS AUTHOR(S)

Identifying a quantum algorithm related to a classical computation is insufficient to justify replacing that computation in existing software. A useful replacement must respect input validation, exactness, ordering, exceptional behavior, and the observable consequences of surrounding code. We study how program analysis and semantic verification can strengthen a fixed large language model (LLM) in identifying candidate computations and designing conditional quantum mappings. QRefactor-Bench supplies source-traceable classical programs, controlled context variants, explicit behavioral obligations, and scoped semantic checks for search and combinatorial optimization. We formulate an evidence-producing workflow that links candidate regions to program facts, represents mapping claims independently of deployment recommendations, and returns reproducible counterexamples to the same model. The evaluation design separates claim correctness from obligation completion and compares direct prediction, self-review, analysis assistance, and semantic feedback under recorded budgets. It further addresses context sensitivity, checker validity, alternative encodings, transfer to unseen source lineages, and the cost of classical fallback. This manuscript is a research draft: empirical results are intentionally unfilled, and the proposed integrated method is not presented as an already validated implementation. The intended contribution is an auditable account of when classical analysis improves quantum mapping design, without equating a plausible plan, a passing local check, or an executable circuit with a correct migration or a quantum advantage.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**; *Software maintenance tools*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: quantum software engineering, program analysis, semantic verification, large language models, selective refactoring, benchmarking

LOCAL PREVIEW: substitute fonts; not a submission PDF.

The standard ACM fonts are unavailable in this local environment. Recompile `main.tex` in a complete TeX environment for final pagination.

1 Introduction

Quantum programming tools expose circuits, operators, and execution interfaces, whereas the maintainer of a classical application starts with program behavior. The maintainer needs to determine *where* a candidate computation occurs, *whether* a supported reformulation is justified, and *how* the surrounding software contract can be retained. These questions arise before committing to a quantum implementation. Retaining the classical computation is a legitimate outcome, including when a conditional quantum mapping can be described.

This problem resembles selective accelerator offloading: a computational kernel is only one component of a replacement. Dependencies, marshaling, selection rules, fallback, and externally visible behavior can dominate both correctness and cost. A graph optimization routine, for example, may return the first optimum according to a specified ordering. An optimizer that returns an arbitrary optimum can solve the mathematical problem while violating the application contract. Similarly, finding one satisfying assignment is not equivalent to returning a complete ordered report, and a failed randomized search does not certify the absence of a solution.

Figure placeholder: overview

Classical program and public contract $\rightarrow$ candidate region $\rightarrow$ structural mapping and obligation ledger $\rightarrow$ conditional plan. Show program-fact and semantic-feedback loops to one LLM; keep adoption, migration validation, and resource evidence separate.

Fig. 1. Proposed workflow and evidence boundaries. The quantum component is a potential target of a conditional plan; the workflow does not require quantum execution.

Existing work provides important foundations. C2|Q translates classical specifications into quantum programs through encoding, deployment, and decoding components [13]. Qiskit HumanEval and QuanBench evaluate quantum code generation using explicit programming tasks and executable checks [3, 12]. QuanBench+ also examines repair after execution feedback [9]. These directions motivate a complementary question: can a fixed LLM reason more reliably about the obligations introduced when a quantum mapping is proposed *inside an existing classical program*? We do not claim that feedback-based generation itself is new.

The central difficulty is connecting a recognized mathematical pattern to the source program's obligations. Input validation, selection order, and intermediate states can invalidate an otherwise plausible mapping. A local formula can pass finite tests while certification and integration remain unresolved.

QREFACTORBENCH supports studying these distinctions. Its development material adapts traceable source tasks into classical programs with functionally relevant context. The supported positive families are deliberately limited to unstructured search and combinatorial optimization. Core-only, hinted-context, and unhinted-context views permit controlled diagnosis. Private reference proposals and semantic checks are kept separate from model-visible inputs. The current reference layer is provisional; it is not described as independently validated ground truth.

Building on this foundation, we propose a workflow that attaches program facts and contract obligations to model claims, checks supported representations, and returns scoped feedback to the same model. Its central hypothesis is that *evidence tied to the source program can improve mapping design beyond the benefit of another model call*. The relevant comparison therefore holds the model fixed and varies the assistance. Cross-model runs assess the reach of a finding; they do not replace this within-model comparison.

The intended paper makes three contributions: (1) a task and benchmark design that separates candidate discovery, structural mapping, practical adoption, and behavioral obligations; (2) a proposed integration of program facts, structured claims, and semantic counterexamples into a fixed-model workflow; and (3) a controlled empirical study of correctness, completion, failure modes, and cost. Only the benchmark infrastructure and a bounded semantic-feedback prototype currently exist. The integrated analysis component and the confirmatory study remain proposed. This draft specifies the study and leaves all empirical result cells blank, rather than supplying hypothetical improvements.

2 Problem and Semantic Boundaries

2.1 Selective quantum opportunity analysis

Let P be a classical program, D its permitted inputs, and C its observable contract. We write $\text{Obs}(P, x)$ for the returned value, relevant effects, exceptions, and ordering observable

Table 1. Evidence required at different stages. Later stages are not inferred from earlier ones. Current experiments primarily concern the first four rows.

Stage	Required output	Distinct unresolved question
WHERE	Candidate spans and dependencies	Is this an admissible region?
WHETHER	Mapping evidence and conditions	Is adoption worthwhile?
WHAT/HOW	Intent, encoding, predicate/objective	Does the formulation match the core?
PLAN	Decoding and obligation disposition	Is the full contract accounted for?
REFACTOR	Implemented hybrid replacement	Does integration preserve behavior?
VALIDATE	Semantic and resource evidence	Is correctness or benefit established?

on input x . A candidate region r is a source region together with the definitions and uses required to interpret it. A mapping proposal M describes an encoding, a search predicate or optimization objective, a quantum procedure, decoding, and integration conditions. A future transformed program $P[M/r]$ would need to satisfy

$$\forall x \in D : \text{Obs}(P[M/r], x) \mathcal{R}_C \text{Obs}(P, x), \quad (1)$$

where $\mathcal{R}_C$ is the relation permitted by the original contract. For deterministic exact outputs this is equality of the required observations. A probabilistic or approximate relation is admissible only when the contract explicitly permits it. No tolerance is inferred from the use of a quantum solver. Equation 1 states the eventual correctness requirement; passing the local checks below does not prove it.

The analysis output is a tuple (r, e, f, M, a, U) : candidate regions r , structural eligibility e , formulation family f , conditional mapping M , adoption recommendation a , and unresolved obligations U . Eligibility takes values {YES, NO, UNCERTAIN}. A YES requires a concrete correspondence between the computation and a supported formulation, rather than only an algorithm name. Adoption depends on additional implementation and resource evidence. Thus $e = \text{YES}$ and $a = \text{REMAIN_CLASSICAL}$ can be consistent. A conditional plan is still required for a structural YES in the prospective protocol.

The historical benchmark combines some of these conceptual stages in serialized task identifiers. We retain those formats when replaying historical material and use the named concepts in this paper to avoid conflating protocol versions. Candidate nomination is also distinct from suitability: a validation scan may be expressible as search yet offer no useful offloading opportunity. Whether it belongs in the intended candidate population requires an explicit reference policy, not an after-the-fact penalty for a model's choice.

2.2 Supported formulations

For search, the model must define a finite domain $\Omega(x)$ and a predicate $g_x(z)$, then explain how marked states correspond to the source computation. Grover's algorithm supplies a search primitive under oracle assumptions [2]. It does not itself supply the reversible oracle, input-loading mechanism, mandated first witness, or no-solution certificate. Those remain explicit plan obligations.

Figure placeholder: contract-example

Show the classical core, minimum-cardinality selection, tuple ordering, and downstream work-list/report dependencies. Highlight where a bitmask-based tie rule changes an observable output.

Fig. 2. Illustrative contract dependencies for the vertex-cover case. The figure is reserved for a source-linked explanation rather than experimental measurements.

For optimization, a QUBO has the form

$$E_x(z) = c_x + \sum_i h_i(x)z_i + \sum_{i < j} J_{ij}(x)z_i z_j, \quad z \in \{0, 1\}^{n_x}. \quad (2)$$

The proposal must relate variables to source choices, encode constraints, specify optimization direction, and decode auxiliary variables. Ising formulations provide established constructions for combinatorial problems [6]. QAOA is a possible approximate optimization procedure [1]; naming it does not certify exact global optimality or preserve a deterministic tie rule. The current positive scope excludes general linear-system and estimation families. Numerical programs can still contain search subregions, which are assessed locally.

2.3 A running contract example

Consider a vertex-cover routine that returns a minimum-cardinality cover and, among ties, the lexicographically smallest tuple of selected vertex indices. Its surrounding program turns the tuple into an ordered work list and a change report. Preserving cardinality without preserving the tuple can therefore alter the external report. Numeric bitmask order and tuple order are different relations and must not be substituted silently.

Let $U(z) = \sum_{(i,j) \in E} (1 - z_i)(1 - z_j)$ count uncovered edges and let $A = 2^n$. One illustrative exact formulation is

$$E(z) = \sum_{i=0}^{n-1} (A - 2^{n-1-i})z_i + PU(z), \quad P = nA + 1. \quad (3)$$

Every selection cost is positive and their total is less than nA . Consequently an infeasible assignment is more expensive than the all-selected feasible assignment. Among feasible assignments, the range of the binary-weight tie term is less than A , so cardinality dominates. Among assignments of equal cardinality, the first differing index dominates all later tie terms and favors inclusion of the earlier vertex. This illustrates why semantic obligations determine the objective. It is not an experimental result, a required reference formula, or a claim that its coefficients are practical on a particular device. Equivalent encodings and smaller valid penalties must remain admissible.

3 Benchmark Construction and Evidence

3.1 Source provenance and functional context

QREFACTORBENCH begins from traceable computations rather than arbitrary code padded with unrelated lines. Each mother case records its source revision, adapted mathematical definition, public contract, classical implementation, and private checking evidence. Source tasks and the newly written application contexts are identified separately. An adaptation

Table 2. Development case inventory. IDs identify cases, not independent sample counts. Scope-control labels remain provisional. Corpus measurements are unfilled.

Case	Source lineage	Classical role	Contract focus
lit-001	C2 Q⟩	Weighted partitioning	Weights, canonical selection, change report
lit-002	C2 Q⟩	Vertex-cover work list	Cardinality, tuple ties, ownership
lit-003	C2 Q⟩	Agenda coloring	Preview versus completion, first solution
lit-004	C2 Q⟩	Compatible records	Filtering, recency, maximum subset
lit-005	QuanBench / QuanBench+	Feature configuration	Locked values, validation order
lit-006	QuanBench	Transfer allocation	Capacity, windows, offsets, ties
lit-007	SupermarQ	Signed preferences	Sign conventions, scores, movement
lit-008	Qiskit HumanEval	Ordered XOR receipts	Full output, order, prefix checksum
lit-009	HPL	Dense balance solving	Pivot selection, exceptions, residuals
lit-010	HPCG	Sparse iterative solving	Initial state, stopping, full trajectory

from a quantum programming task is not presented as an original production application supplied by its upstream authors.

Table 2 lists development cases and obligations. Shared mathematical structure and upstream tasks prevent treating every directory or narrative as an independent problem family.

C2|Q⟩, QuanBench, QuanBench+, Qiskit HumanEval, and SupermarQ supply task or objective provenance [3, 9, 11–13]. HPL and HPCG supply numerical algorithm references [4, 5]. These uses do not reproduce the upstream systems’ complete workloads or official performance measurements. Circuit resources and representation levels can be studied using ideas from MQT Bench [8], but resource extraction does not establish the semantic correctness of the enclosing application.

3.2 Controlled input views

For each mother case, view A contains the computational core; view B contains the full classical program plus a source-location cue; view C contains the same full program without that cue. The B/C pair is byte-identical after removal of the designated cue. This makes B/C the clean contrast for location assistance. A/B changes both context and the obligations visible to the model, so differences cannot be attributed solely to search difficulty. An analysis of core obligations shared across A and B is reported separately from full-contract completion.

Context must affect the public contract, as assessed through dependency and mutation checks. Program size, branches, obligations, and dependency distance are descriptive covariates; difficulty is measured rather than assumed.

Public input packages contain only allowlisted source and task material. Private reference proposals, tests, previous responses, and diagnostic conclusions are excluded. Source notices preserve attribution and licensing, so the benchmark does not claim perfect source anonymity. Any redaction or renaming experiment must retain legally required notices and use a separately versioned input.

3.3 Reference construction and uncertainty

The reference process begins with a behavioral inventory: input domain, returned values, ordering, exception behavior, relevant state, and downstream uses. Reviewers then identify possible regions and admissible formulations. They record whether a concrete mapping is supported, contradicted, or unresolved and state the scope of each judgment. Multiple valid regions and different formulation families may be accepted when their obligations are met.

The current materials contain AI-assisted proposals and coordinator review; these are not independent gold annotations. For a confirmatory release, separate reviewers should inspect the original program and contract before seeing model answers. The proposed process records identity or role, expertise, independence, initial judgments, disagreements, and adjudication. Difficult cases remain uncertain when evidence is insufficient. The final inclusion policy is a human scientific decision, recorded before confirmatory responses are collected.

Correctness and completion occupy different axes. A formula explicitly supplied but refuted is *provided and contradicted*. A response that honestly defers the tie-breaking construction is *unresolved and incomplete*. Neither should be converted into the other by a keyword score. Obligations cover core correspondence, encoding, exact selection or certification, decoding, and context preservation. Each record binds the original response passage, its interpretation, the applicable obligation, evidence, and remaining uncertainty.

3.4 Local checkers and their scope

Trusted checkers compare a supported claim against a definition of the classical task. For a QUBO, a bounded exhaustive checker enumerates assignments and checks *every* globally optimal decoded state; accepting one convenient ground state would miss invalid tied optima. For a search predicate, it compares marked states with the task relation and separately handles selection order, illegal encodings, and no-solution behavior. Numeric checks preserve the declared arithmetic model and distinguish permitted inputs from reachable intermediate states.

The checker suite includes known-correct constructions, equivalent encodings, and deliberate semantic errors. Examples include reversed optimization direction, missing constraints, wrong tie order, discarded weights, and omitted iteration trajectories. These controls validate an evaluator; they are never counted as model errors or model repairs. A checker can also fail: equality of the zero-energy set alone, for example, does not establish that feasible states minimize the energy. The validation record must include extremum checks and retain the failure that motivated a correction.

Each verdict names its representation, tested domain, arithmetic, and executable evidence. Unknown representations and infrastructure errors stay distinct from semantic refutations. In the existing workflow, some natural-language claims require a reviewer's structured transcription. This is not an automatic judge of arbitrary prose. The experimental design

Figure placeholder: evidence

Four layers: public source/contract; private reference and admissible alternatives; model claim with exact passage; scoped check and unresolved obligations. Draw a boundary between development feedback and evaluator-reserved final tests.

Fig. 3. Provenance and evaluation separation. A verdict is attached to a specific claim and domain, rather than promoted to whole-program correctness.

measures that transcription burden and reports what fraction of claims can actually be checked.

4 Proposed Method: Source Facts and Semantic Feedback

4.1 Design hypothesis and implementation boundary

The proposed method strengthens the same LLM by supplying evidence that it can use to revise a candidate and its mapping. It combines a source analysis stage, an obligation ledger, claim checking, and bounded revision. This is an integration proposal, not a claim that the full architecture is already implemented. The existing infrastructure supports source coordinates, structured predictions, private review records, and trusted local checks. A separate prototype accepts structured vertex-cover QUBO templates and returns development-test feedback. Automatic extraction of full program contracts, general numeric-state analysis, and production-program transformation remain outside that implemented prototype.

The technical hypothesis is specific: errors in a mapping often arise because a mathematical pattern is detached from its source preconditions or observable selection rules. A dependency fact or a reachable counterexample can correct that detachment. This hypothesis is narrower than assuming that more tools or a longer prompt improve the entire task. The ablations in Section 5 are designed to isolate which evidence, if any, helps.

4.2 Candidate-centered program facts

Given a nominated region, the proposed analysis constructs a bounded dependency slice covering live inputs, state updates, guards, and uses of its result. A fact has a source location, an analysis origin, an assumption set, and a confidence status. Examples include a loop's iteration order, an explicit finite-value check, the use of a tuple in an ordered report, and a caller-selected iteration budget. The model sees these facts with their limitations; the analyzer must not label a heuristic inference as a theorem.

For dynamic Python features outside the supported analysis, the method returns an unresolved dependency rather than assuming purity or a closed call graph. Alias-sensitive updates and exception paths require conservative treatment. Facts established at the program entry are not silently lifted to every later program point. In particular, finite numeric inputs do not imply that intermediate floating point operations remain finite. A dynamic witness can demonstrate a reachable state; a collection of successful traces cannot prove that an unobserved state is unreachable.

An obligation ledger links these facts to the proposal. For each obligation it stores the required behavior, source anchors, proposed implementation or proof, available check, and status. The ledger is not a hidden answer key: public facts may be derived from the input, whereas evaluator-only reference judgments remain withheld. The full method is compared

with both an identical prompt-only ledger and a fact-assisted workflow to avoid crediting the presentation format alone.

4.3 Checking mapping claims

The model first supplies the original conditional plan. Supported parts may then be expressed in a restricted claim representation: finite predicates, coefficient templates, variable mappings, optimization direction, decoding, and preconditions. For a plan originally written in prose, transcription is logged and checked against the source passage. If a reviewer must invent missing coefficients or assumptions, that addition is a separate proposal and not credited to the model.

The vertex-cover prototype illustrates the representation boundary. It interprets a whitelist of arithmetic expressions, builds a quadratic polynomial, and uses exact rational arithmetic. Templates describe a family of graphs; they cannot select answers by looking up final-test instances. Variable permutation, bit complementation, constant offsets, and appropriate scaling may yield equivalent solutions and must be accepted. A formula outside the representation’s expressivity receives a representation outcome, not a universal judgment that no valid mapping exists.

For each supported claim q , the checker returns

$$V(q) = (s, \mathcal{D}, \alpha, w), \quad s \in \{\text{supported, refuted, unknown}\}, \quad (4)$$

where $\mathcal{D}$ is the checked domain, α the assumptions, and w an evidence record. A refutation includes a witness and the violated obligation; finite support includes the actual test boundary. A universally quantified correctness claim requires a proof beyond a finite successful test run.

4.4 Bounded feedback to the same model

Feedback exposes the first failing development instance, the claimed output or decoded optimum, the required observation, and the violated obligation. It does not expose a replacement reference formula or any final-test result. When all development checks pass, feedback says only that the declared checks passed. The same model revises its whole plan within a fixed call budget, retaining the ability to narrow a claim, state an unresolved obligation, or recommend keeping the original classical routine.

Table 3 describes the proposed orchestration. The feedback loop resembles the candidate/verification separation of counterexample-guided synthesis [10], but an LLM proposal plus bounded testing does not inherit synthesis completeness or termination guarantees. The workflow ends on the predetermined budget, not only when a favorable answer appears.

4.5 Reachable states and classical fallback

A useful diagnostic concerns a proposed first-maximum predicate. Over ordinary totally ordered finite values, “no smaller than every candidate, with no earlier equal value” can characterize a first maximum. With NaN values, these comparisons do not reproduce a sequential incumbent-update rule. A counterexample is relevant to a program only if the state is permitted or reachable from a permitted input. The planned numeric analysis therefore binds a witness to the original program point and execution trace instead of testing an arbitrary impossible array.

This distinction also constrains feedback scope. Refuting a predicate does not refute every conditional plan that mentions it. A plan may detect an exceptional state and fall back to the classical routine. Review must determine whether that guard is actually supplied,

Table 3. Proposed bounded workflow. Evaluator-reserved checks never produce feedback within a measured run.

Step	Operation and retained evidence
Input	Freeze public program, contract, model configuration, and budgets.
Propose	Save raw candidate regions, eligibility, mapping, recommendation, and open obligations.
Analyze	Derive source facts with assumptions and source anchors; record unsupported constructs.
Check	Validate supported claims against development checks; retain scope and witnesses.
Revise	Send allowed facts and feedback to the same model; save every revision.
Stop	End at the fixed budget or protocol stopping condition, retaining failures.
Evaluate	Run reserved checks and blinded review once on the designated final artifact.

occurs before the incorrect operation, and preserves the full contract. A vague promise to “verify and fall back” is incomplete until the verification condition and fallback behavior are specified. Conversely, an exact but expensive fallback may preserve correctness while eliminating any practical benefit. The method records these dimensions separately.

4.6 Resources and adoption

Structural mapping and practical adoption use different evidence. A resource ledger records state preparation, reversible oracle construction, ancilla cleanup, logical and decomposed circuit resources, repeated measurements, classical optimization, data transfer, checking, and fallback. Unknown terms remain unknown. Any circuit counts name the representation and compilation target; they are not used as an unexplained profitability score.

The method never turns simulator wall time into a quantum speedup claim. For this paper, its own usefulness is measured as improved analysis or mapping quality and the classical/model cost of obtaining it. An end-to-end advantage study would need implemented hybrid programs, appropriate classical competitors, hardware assumptions, and a separate experimental protocol.

5 Evaluation Design

This section specifies a proposed confirmatory study and distinguishes it from available exploratory artifacts. The final sample size, model snapshots, budgets, reference policy, and split have not been approved or frozen. Writing the protocol does not mean these experiments have been run. The design emphasizes interpretable contrasts rather than a single overall leaderboard.

5.1 Research questions

- RQ1: Baseline limitations.** Which obligations do direct LLM predictions recognize, supply, and satisfy, and which failures are hidden by coarse labels?
- RQ2: Context and localization.** How do location cues and surrounding behavior affect candidate identification and mapping quality?
- RQ3: Same-model improvement.** Does source analysis plus semantic feedback improve checked claim correctness and obligation completion over direct prediction and self-review using the same model?

Table 4. Corpus and review report to complete after the population is fixed. Blank cells mean unfilled, not zero or an observed unknown.

Partition	Mothers	Lineages	Views	Reviewed
Development-exposed				
Method validation				
Confirmatory holdout				
Search stratum				
Optimization stratum				
Scope / uncertain controls				

- RQ4: Mechanism.** Which benefits depend on source facts, explicit obligation tracking, counterexample content, or simply additional inference budget?
- RQ5: Trust and transfer.** Are the evaluators discriminating and tolerant of equivalent representations, and do any gains transfer to unseen source lineages and behavior-preserving variants?
- RQ6: Cost and practical limits.** What model, analysis, checking, review, and fallback costs accompany any gain, and when should the method abstain?

5.2 Population, grouping, and leakage control

The current corpus has already informed prompts, checks, and failure analysis. It is therefore development-exposed. Its unrun input views are not an independent holdout because they share a mother case with observed responses. The proposed confirmatory population comprises newly reviewed source lineages, selected for scope and contract coverage before inspecting the evaluated models' answers. Corpus sizes and group counts remain blank in Table 4.

All views, renamings, parameter changes, and contract variants of one mother case stay in the same partition. Shared upstream tasks and substantially shared kernels form larger lineage groups. The split uses the larger dependency group where appropriate. Selection and exclusions are recorded with reasons; instances are not retained because they happen to separate models. Test generation and checker changes after seeing responses are explicitly post-hoc diagnostics, never retroactively part of the confirmatory score.

The model runtime receives a frozen input allowlist and no private references, prior responses, or final tests. For a tool-assisted run, accessible tool outputs are separately specified, and arbitrary repository access is prohibited. A fresh output directory, immutable raw responses, hashes, and event logs support replay. Hashes establish artifact identity rather than truth or evaluator security. Public pretraining contamination cannot be ruled out by local access restrictions; source exposure and robustness analyses address it only partially.

5.3 Experimental conditions and budget controls

The principal unit of comparison is a fixed model configuration on the same mother case. Direct prediction is compared with self-review, analysis-assisted revision, semantic-feedback revision, and their combination (Table 5). The self-review baseline is motivated by work on iterative self-feedback [7], but its exact prompt and budget are frozen for this task rather than assumed to reproduce a published implementation.

For the main revision experiment, independent initial drafts are saved and cloned into the revision arms. Each branch receives the same initial answer, cannot see other branches,

Table 5. Proposed within-model conditions. All revision arms share the initial draft in the paired study. A common contract-oriented prompt is used throughout.

Arm	Source analysis	External semantic evidence	Purpose
D: Direct	None	None	One-shot reference
S: Self-review	None	None	Extra-call control
A: Analysis	Source facts	None	Fact contribution
V: Verification	None	Development witness or finite-pass scope	Feedback contribution
AV: Combined	Source facts	Development witness or finite-pass scope	Proposed integration

and is allowed the same number of revision calls. Arm order is balanced within each model and time block. A separate pre-generation analysis contrast measures whether facts help the *initial* proposal; it is not mixed with the shared-initial-draft repair comparison.

Equal calls are not equal computation. We record prompt and completion tokens, reasoning tokens when available, cache use, wall time, tool time, and monetary cost when meaningful. The primary configuration uses matched call caps and declared output limits. Sensitivity analyses use a predeclared total token cap and a wall-time cap; realized usage remains visible. Padding prompts with irrelevant text would not establish equivalent useful computation. Different providers' effort labels are not treated as equivalent budgets.

Models are chosen to assess whether the mechanism generalizes beyond one model, with exact identifiers, access path, date, revision when exposed, and configuration recorded. Unknown temperature, seed, or server snapshot stays unknown. Existing multi-provider pilot runs have different scaffolding and budgets and are reported as exploratory material, not a matched capability ranking. The new model count, replication count, timeout, and total call budget are deliberately unfilled: models _____; replicates per mother _____; timeout _____; total calls _____.

5.4 Outcome definitions

We report a vector of outcomes rather than a weighted aggregate. Delivery status distinguishes a valid final answer, schema failure, output-budget truncation, infrastructure failure, and not-run slot. All planned slots remain in the denominator inventory. A truncated answer supplies no successful plan, but is not invented evidence of a false mathematical claim.

Localization and recognition. Exact source-region agreement and merged-line intersection over union are descriptive localization diagnostics. The final scientific interpretation of alternative boundaries uses the frozen reference policy. Structural eligibility, intent, family, support, practical suitability, and adoption are recorded separately. For unresolved reference labels, report coverage and abstention without treating uncertainty as a negative class. If a stratum has no reviewed negative references, do not report negative-class accuracy for it.

Claim correctness and obligation completion. For an applicable obligation o in case i , let p_{io} indicate supplied content and let v_{io} take supported, refuted, or unresolved values. With

the reference obligation set O_i fixed independently of the answer, report

$$C_i = \frac{\sum_{o \in O_i} p_{io}}{|O_i|}, \quad S_i = \frac{\sum_{o \in O_i} \mathbf{1}[v_{io} = \text{supported}]}{|O_i|}, \quad F_i = \frac{\sum_{o \in O_i} \mathbf{1}[v_{io} = \text{refuted}]}{|O_i|}. \quad (5)$$

Also report the unresolved share and the evidence scope for each obligation. These are proposed descriptive coverage measures, not a replacement for the existing evaluator or a whole-plan correctness score. Multiple assertions within one obligation retain separate evidence records; one supported assertion cannot erase a contradiction or discharge unrelated requirements. The obligation-level aggregation rule must be frozen before use. A conditional accuracy among checked claims is accompanied by its checking coverage to prevent selective verification from appearing comprehensive.

Revision transitions. On each applicable checked claim or explicitly scoped local task, retain the before/after transition: refuted to supported, supported to refuted, unresolved to supported, unresolved to refuted, or unchanged. A claim removed from the answer may reduce false assertions while decreasing completion; it is not automatically a successful repair. Repair yield is reported on genuinely refuted initial drafts and alongside the full-population change. If no initial error occurs, conditional repair capability is unmeasured rather than perfect.

Context obligations and fallback. Report validation order, exact witness or optimum selection, no-solution handling, decoding, side effects, and caller-visible trajectories separately. A statement that a classical fallback exists is not an executed integration test. Where a reviewed implementation is available, report guard coverage, fallback frequency, and cost. Otherwise these remain plan obligations. Whole-program contract preservation and quantum resource feasibility are never imputed from local checks.

5.5 Experiments and ablations

E1: Direct prediction and failure characterization (RQ1). Run the unhinted-context condition with first attempts preserved. Characterize delivery, region nomination, structural judgments, planning coverage, semantic refutations, and incomplete obligations. Compare coarse label agreement with content-level evidence on the same responses. Code failures by mechanism and report uncertainty rather than forcing all cases into a single error class.

E2: Context and location cues (RQ2). Run paired A/B/C views with new sessions and a balanced order. Compare B/C for location assistance, and compare shared core obligations across A/B. Report any change in candidate over-nomination on scope controls. Stratify descriptively by dependency distance, branch structure, and ordering obligations. The data may show no additional difficulty; that is a valid result.

E3: Paired revision (RQ3). Compare D, S, A, V, and AV using shared drafts for the revision arms. Measure supported obligations, refuted claims, unresolved obligations, regression, and completion under reserved checks. The main planned contrasts are AV versus S and V versus S, with A versus S and AV versus V isolating the added analysis. Select the confirmatory contrast before running the study; report all remaining contrasts as secondary. Include correct initial drafts so that regressions and ceiling effects remain visible.

E4: Component and information ablations (RQ4). Remove source facts, remove the obligation ledger while retaining task content, replace a witness with a verdict-only message, and omit the violated-obligation explanation. Each comparison changes one factor within

its parent condition. Add the separate pre-generation fact condition and budget-matched self-review. No arm receives final-test answers. Assess whether richer feedback helps repair, merely elicits longer plans, or causes unjustified confidence. All ablations are proposals pending implementation and authorization.

E5: Evaluator validity and transcription (RQ5). Test independent task definitions against known-correct, equivalent, and faulty constructions. Cover variable permutation, bit complement, constant energy shifts, optimization-preserving scaling, and invalid-state extrema. Audit model-passage to claim transcription independently, including ambiguous formulas. Measure unsupported representations, disagreement, transcription time, and false acceptance or rejection on controls whose expected behavior is independently established. Never report a control mutation kill rate as model improvement.

E6: Reachability and behavioral robustness (RQ5). Vary input order, tied optima, empty/no-solution cases, invalid inputs, numerical boundaries, and caller budgets where the public contract allows them. Distinguish behavior-preserving transformations from contract-changing variants. For numerical counterexamples, provide a path from valid inputs to the tested intermediate state. Compare checking a local expression alone with checking it under the source-state conditions. These tests do not add unsupported quantum families.

E7: Transfer and model sensitivity (RQ5). Freeze the method on development lineages and evaluate unseen lineages without updating prompts, facts, or checkers from their responses. Report within-model gains separately for search, optimization, and unresolved controls. Repeat the same assistance contrasts across models, preserving provider-specific settings. Leave-one-lineage-out development analysis can guide uncertainty estimates, but it is not relabeled as an untouched final holdout.

E8: Efficiency and adoption limits (RQ6). Report model calls, tokens, wall time, analysis time, exact-checking time, peak memory when measured, human review effort, and fallback cost. Plot quality against the recorded call/token/time budgets only after results exist. Vary bounded enumeration size to expose checker scaling. Preserve timeout and unsupported outcomes. If circuit resource data are collected, report them separately with representation and backend assumptions; no hardware advantage is inferred.

5.6 Statistical analysis and stopping rules

The independent sampling unit is the mother problem or, when shared provenance requires it, the larger lineage group. Repeated drafts, context views, obligations, and individual checker instances are nested observations. For a per-mother metric y and paired arms a, b , first average within the fixed replicate set to obtain $d_i = \bar{y}_{i,a} - \bar{y}_{i,b}$. Report the distribution of d_i and the lineage-weighted contrast, giving each independent lineage equal weight. Do not multiply a formula's test instances by response count to enlarge sample size.

The proposed interval procedure resamples whole independent lineage groups while retaining all paired arms and nested observations. The confidence level, bootstrap replications, minimum useful effect, and required interval precision are fixed before execution. If groups are too few for reliable interval inference, report the paired observations descriptively and state that limitation. A power or precision analysis may use development variability, but must not promise that a convenient case count is sufficient. Confirmatory multiplicity correction applies to the predeclared family of contrasts; exploratory strata remain labeled as such.

Missing delivery contributes to the fixed-slot outcome inventory and cannot be removed to improve reported success. Analyses restricted to checkable responses state their reduced

Table 6. Baseline results by fixed model configuration. Expand one row per evaluated configuration; retain the denominator of all scheduled requests.

Configuration	Requests	Valid	Label agree.	Supported	Refuted
Model/configuration 1					
Model/configuration 2					
Model/configuration 3					
Model/configuration 4					

coverage. No mathematical correctness label is imputed for missing output. Collection stops at the fixed budget or a documented infrastructure stop rule. There are no selective retries, extra repetitions after disappointing results, or model substitutions within the registered experiment.

5.7 Reproducibility and research use of AI

The replication record should include protocol and source hashes, raw provider responses, timing and usage, tool events, parsing decisions, claim transcriptions, review provenance, environment locks, and derived-table scripts. Development feedback and reserved evaluation inputs are stored separately. Anonymous reviewer access must be tested before submission; this draft does not claim a deposited artifact.

AI tools have assisted development of case proposals, code, checking infrastructure, review drafts, and this methodological draft. Models are also the experimental subjects. These roles are distinct: an AI-proposed reference is not an independent human annotation, and a generated checker is not trusted solely because it runs. The final methods record must identify the tools, versions where available, artifacts affected, human oversight, and validation for each research role. No independent expert agreement is inferred from an unattributed review response.

6 Results: Reserved Reporting Structure

All empirical cells in this section are intentionally blank. Empty cells are not zero, measured uncertainty, or omitted failures. They reserve the location of verified results once an approved experiment and review are complete. The text specifies the analysis to report and does not anticipate its direction.

6.1 Baseline diagnosis and context (RQ1–RQ2)

Table 6 separates delivery, reference agreement, and checked content. Report reference uncertainty and checking coverage alongside any apparent ceiling in coarse recognition.

The context analysis should distinguish better localization, newly visible obligations, and irrelevant nominations, using source-linked examples.

6.2 Paired enhancement and ablation (RQ3–RQ4)

Table 8 pairs supported content with completion and refutations. Deleting false claims and supplying correct constructions are different outcomes. Effect estimates must follow the frozen contrasts.

The existing structured-QUBO prototype and the original conditional-plan task must be reported separately. Supplying a core and its exact contract changes the task relative to discovering obligations in an unhinted complete program. A prototype with no erroneous

Table 7. Context contrasts on paired mother cases. A/B scores for content refer only to shared core obligations; B/C isolates the location cue.

Outcome	A	B	C	Paired B–C
Valid delivery / scheduled				
Exact region agreement				
Merged-line overlap				
Shared core support				
Core obligation completion				
Unresolved references				

Table 8. Same-model comparison. Each table instance refers to one model and one frozen task protocol. Supported claims retain their bounded evidence scope.

Outcome	D	S	A	V	AV
Scheduled drafts / branches					
Valid final responses					
Supported obligation share					
Refuted obligation share					
Unresolved obligation share					
Supplied obligation share					
Refuted → supported					
Supported → refuted					
Actual counterexample exposures					

Table 9. Ablation and budget comparisons. Each difference is paired within model and lineage; effect estimates and uncertainty remain unfilled.

Contrast	Support change	Completion change	Interval
AV versus S			
AV versus V (analysis contribution)			
V versus S (external verification)			
Full versus no obligation ledger			
Witness versus verdict only			
Explained witness versus raw witness			
Facts before versus after initial draft			
Matched token / time sensitivity			

initial drafts cannot measure counterexample-driven repair, regardless of the number of final instances checked. The historical record must retain its own protocol and any ceiling outcome when results are eventually inserted; it must not be presented as confirmatory evidence for the new integrated design.

Table 10. Evaluator and generalization evidence. Control outcomes are not model scores. Each transfer row is grouped by source lineage.

Population / check	Eligible	Assessed	Outcome
Known-correct controls accepted			
Equivalent encodings accepted			
Faulty controls rejected			
Unsupported representations			
Independent transcription agreement			
Reachable-state witness reproduction			
Unseen search lineages: paired gain			
Unseen optimization lineages: paired gain			
Uncertain controls: resolution / abstention			

Table 11. Cost report for a fixed model and task population. Empty entries must be filled from recorded usage, not inferred from model names or token caps.

Cost component	D	S	A	V	AV
Model calls					
Input / output tokens					
End-to-end wall time					
Source analysis time					
Development checking time					
Reserved checking time					
Peak tool memory					
Review / transcription time					
Fallback frequency / cost					

6.3 Evaluator trust and transfer (RQ5)

Table 10 separates controls from model predictions. Report false acceptance, false rejection, equivalent encodings, and transcription disagreements; a precise checker cannot resolve ambiguous source prose.

Two planned qualitative analyses connect outcomes to mechanisms. The selection analysis traces feasibility, primary objective, and canonical ordering through the decoder and downstream report. The numeric analysis traces a permitted input to a problematic intermediate comparison, then examines whether the model specified a sufficient guard or fallback. Each analysis should show the initial claim, evidence returned, revision, and reserved-test scope, including unsuccessful repairs. Examples are explanatory and do not enlarge the independent sample size.

6.4 Cost and scaling (RQ6)

Table 11 reports complete and incremental costs, including the shared initial draft. Explain cache and provider accounting; separate human review from automated latency.

No empirical plot is drawn before data exist. The intended plots are paired per-lineage changes with uncertainty, quality versus recorded inference budget, and checking time versus

enumeration size. They must be generated from the final result tables without synthetic curves. The source package includes an experiment matrix and result-field inventory to support later data insertion.

7 Discussion and Threats to Validity

7.1 What an improvement would establish

Improved mapping evidence would support a bounded analysis result, not production refactoring or quantum advantage. Reduced false confidence must be interpreted with completion and abstention. Exact fallback can preserve behavior without providing computational benefit.

A model experiment does not establish developer productivity; that requires a separate user study. A bounded checker may nevertheless serve as a design debugger without scaling to production inputs.

7.2 Construct validity

Keeping classical code can be justified without demonstrating mapping ability. Separate structural, practical, and content outcomes avoid conflating these judgments.

Reference labels and candidate boundaries remain a material source of uncertainty. Optimization, search, and validation scans can overlap in legitimate ways. A different family or wider source region is not automatically an error. Independent review and explicit acceptance criteria are required to interpret these choices. The reference obligation inventory itself must not expand only for a favored method's longer answers.

Finite tests provide bounded support. Exhausting bitstrings for selected graphs does not exhaust all graphs or all contexts. Numeric fixtures using exactly representable values do not prove arbitrary floating point equivalence. A local predicate may be refuted without invalidating a complete guarded plan. Reported outcomes must retain these boundaries, including the difference between a valid template and a fully implemented migration.

7.3 Internal validity

Calls, prompts, providers, tool latency, and caches can confound comparisons. Shared drafts, budget analyses, and time-blocked execution reduce these effects. Unobserved service-side model changes still prevent claiming immutable weights.

The reference, interpreter, and checker may share mistakes. Independent task definitions, equivalence controls, fault injection, and original-program comparisons provide complementary checks. They do not prove the evaluator correct. Manual transcription adds another error channel; passage binding and independent review are therefore part of the planned evidence, not clerical details.

Development feedback can leak into evaluation through prompts, case selection, checker revisions, or informal knowledge of test outputs. Separating whole lineages and freezing the protocol limits direct leakage. Post-hoc counterexamples remain valuable diagnostic evidence but must retain their post-hoc designation. No repeated use of a final test is described as a fresh holdout assessment.

7.4 External and conclusion validity

Source-derived synthetic contexts provide control and traceability but do not represent arbitrary industrial repositories. The limited positive families, Python front end, small exact checks, and chosen expression language restrict generalization. Program analysis assumptions

may fail on dynamic dispatch, reflection, complex libraries, concurrency, or native extensions. These cases must remain unsupported rather than be counted as correctly handled.

Small or correlated source collections produce uncertain effect estimates. Replicate calls, obligations, and test instances cannot substitute for independent program lineages. A ceiling experiment cannot demonstrate repair effectiveness, and absence of a measured gain does not establish that counterexamples never help. The planned reporting preserves null results, regressions, and incomplete delivery. Any later paper-level claim must be narrower than the evidence that actually survives independent review.

8 Related Work

Classical-to-quantum development. C2|Q provides a modular path from classical specifications to encoding, quantum execution, and decoding [13]. It is an especially relevant comparison because it begins with classical problem descriptions. Our intended emphasis is on candidate regions inside surrounding software, the distinction between a conditional mapping and adoption, and source-linked behavioral obligations. Some development cores derive from its dataset, so the source dependence is explicit. This paper does not assert that prior work lacks every form of semantic checking. Any future direct system comparison must align task scope, access, and contracts.

Quantum code-generation benchmarks. Qiskit HumanEval evaluates generated Qiskit programs on curated tasks [12]. QuanBench combines functional checking with quantum semantic evaluation [3]. QuanBench+ extends evaluation across frameworks and includes feedback-based repair [9]. The proposed distinction is source-contract evidence for the same model and separate measurement of claim correctness and completion, rather than the addition of an execution-feedback loop alone.

Quantum algorithms and workload evaluation. Grover search, QAOA, and Ising formulations provide the algorithmic and mathematical basis of the supported families [1, 2, 6]. Their premises must be connected to source behavior rather than assumed from a problem label. SupermarQ emphasizes application-oriented quantum benchmarking [11], while MQT Bench supports multiple circuit abstraction levels [8]. These motivate explicit evaluation boundaries and representation-aware resource reporting. Neither a device benchmark score nor a circuit count is substituted for preservation of a classical application’s observations.

Feedback and synthesis. Self-Refine studies iterative improvement using model-generated feedback [7]. Counterexample-guided synthesis separates a proposed candidate from validation and uses failures to guide subsequent candidates [10]. The planned workflow adopts a related separation between model proposal and external evidence, without claiming a general synthesis algorithm or inheriting formal guarantees. Its research question is whether program-specific facts and scoped counterexamples improve quantum opportunity analysis beyond matched self-review. Establishing that contribution requires the planned controlled study; the current local prototype alone is insufficient.

9 Conclusion

We investigate how source contracts can guide LLM-based quantum opportunity analysis. This draft combines traceable cases, separate correctness and completion evidence, and a proposed same-model workflow using program facts and semantic feedback. The study isolates context, inference budget, method components, transfer, and cost. Results remain unfilled; enhancement effectiveness and its scope are questions for the proposed evaluation.

Data Availability

The research artifact currently exists as a local development repository. An anonymous reviewer-accessible replication package has not yet been deposited. The intended package contains versioned public inputs, source and license records, protocols, raw responses, scoped annotations, trusted checkers, and regeneration instructions, with development feedback separated from final evaluation material. An anonymous access location and a statement of the authors' intended release policy will be supplied here before submission. No public availability or independent artifact evaluation is claimed in this draft.

References

- [1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A Quantum Approximate Optimization Algorithm. arXiv:1411.4028. doi:10.48550/arXiv.1411.4028
- [2] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. arXiv:quant-ph/9605043. doi:10.48550/arXiv.quant-ph/9605043
- [3] Xiaoyu Guo, Minggu Wang, and Jianjun Zhao. 2025. QuanBench: Benchmarking Quantum Code Generation with Large Language Models. arXiv:2510.16779. doi:10.48550/arXiv.2510.16779
- [4] HPCG Benchmark Project. n.d.. Official HPCG Benchmark Source Code. <https://github.com/hpcg-benchmark/hpcg>
- [5] HPL Project. n.d.. HPL Algorithm. <https://www.netlib.org/benchmark/hpl/algorithm.html>
- [6] Andrew Lucas. 2013. Ising Formulations of Many NP Problems. arXiv:1302.5843. doi:10.48550/arXiv.1302.5843
- [7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651. doi:10.48550/arXiv.2303.17651
- [8] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2022. MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing. arXiv:2204.13719. doi:10.48550/arXiv.2204.13719
- [9] Ali Slim, Haydar Hamieh, Jawad Kotaich, Yehya Ghosn, Mahdi Chehimi, Ammar Mohanna, Hasan Abed Al Kader Hammoud, and Bernard Ghanem. 2026. QuanBench+: A Unified Multi-Framework Benchmark for LLM-Based Quantum Code Generation. arXiv:2604.08570, version 2. doi:10.48550/arXiv.2604.08570
- [10] Armando Solar-Lezama. 2008. *Program Synthesis By Sketching*. Ph.D. Dissertation. University of California, Berkeley. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html>
- [11] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. 2022. SupermarQ: A Scalable Quantum Benchmark Suite. arXiv:2202.11045. doi:10.48550/arXiv.2202.11045
- [12] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models. arXiv:2406.14712. doi:10.48550/arXiv.2406.14712
- [13] Boshuai Ye, Arif Ali Khan, Teemu Pihkakoski, Peng Liang, Muhammad Azeem Akbar, Matti Silveri, and Lauri Malmi. 2025. C2—Q₂: A Robust Framework for Bridging Classical and Quantum Software Development. arXiv:2510.02854. doi:10.48550/arXiv.2510.02854